



1 Problembeschreibung

Konsolenbasiertes System zur Verwaltung und Buchung von Flügen.

- **Verwalten:** Flugzeuge und Flüge, reale Flughäfen, Fluggesellschaften
- **Buchung:** Suchen und Buchen, Sitzplatz- und Gepäckwahl, Umbuchung und Storno
- **Einschränkungen:** nur Direktflüge, Buchungen am Schalter, feste Preiskategorien

2 Teamorganisation

- **Entwicklung:** Marlon, Alexander
- **Dokumentation:** alle Teammitglieder
- **Präsentation:** Ouail, Neele, Nikola, Enrico
- **Protokolle:** Enrico
- **Projektleitung:** rotierend
- **Entscheidungen:** demokratisch per Abstimmung
- **Meetings:** zweimal wöchentlich, durchschnittlich 120 Minuten, in Präsenz oder über Discord
- **Werkzeuge:** WhatsApp und Discord (Kommunikation), GitHub (Projektmanagement und Datenarchiv), Eclipse (Entwicklung)

3 Lösungsalternativen und Entscheidungen

- **Architektur:** zentrale Klasse vs. spezialisierte Service-Klassen – gewählt: spezialisierte Service-Klassen (klar abgegrenzt, leichter wartbar)
- **Datenstruktur:** Liste vs. HashMap – gewählt: HashMap für Buchungen (direkter Zugriff per Schlüssel), Liste für Flüge und Passagiere (vollständiger Durchlauf)

4 Funktionsweise

- **Programmstruktur (Pakete):** model (Datenobjekte), service (Programmlogik), repository (Datenhaltung), interfaces (einheitliche Struktur), exceptions (Fehlerbehandlung), UI (Konsolenoberfläche)
- **model-Klassen:** Flug, Flugzeug, Sitz, Passagier, Buchung
- **Services:** FlugSuch-, Buchungs-, Preis-, Storno- und Umbuchungs-Service
- **Datenerhaltung:** Zusammenfassung der Repositories in einem serialisierbaren Objekt; Speicherung beim Beenden, Laden beim Start
- **Repositories:** speichert die Programmdaten wie Flüge, Passagiere, Flughäfen und Buchungen
- **Interfaces:** Legen fest, welche Methoden die Service- und Repository-Klassen anbieten müssen
- **Exceptions:** Fangen typische Fehlerfälle ab, identifizieren sie und geben diese verständlich aus
- **Programmstart und KonsolenUI:** beim Start werden die Anwendungsdaten geladen, die benötigten Repositories und Services initialisiert und verknüpft; die KonsolenUI steuert anschließend das Menü, verarbeitet die Nutzereingaben und verbindet so Nutzer und Programmkomponenten
- **Ablauf einer Buchung:** Flug suchen → Flug auswählen → Sitz wählen → Gepäckanzahl → Bestätigung

5 Besonderheiten des Programms

- Differenzierte Wahl der Datenstruktur (HashMap und Liste)
- Interfaces als Vorlagen für Service- und Repository-Klassen
- Eigenes Paket für Exceptions mit spezifischen Klassen je Fehlerfall
- Verwendung von Enums für vorgefertigte Auswahlmöglichkeiten

6 Fazit

- Praxisnahe, bewusst vereinfachte Abbildung einer Flugbuchung
- Modularer, objektorientierter Aufbau der Kernfunktionen
- Umsetzung fachlicher Anforderungen in eine klare Architektur
- Bewusste Vereinfachungen zugunsten von einer übersichtlicheren Struktur
- Fundament für mögliche zukünftige Erweiterungen

Mögliche Erweiterungen: Unterstützung von Umstiegen (Nicht-Direktflüge), grafische Benutzeroberfläche (GUI), automatisierte Erstellung regelmäßiger Flüge